

Guía del Backend — Krypton E-commerce

Guía didáctica de **cómo funciona el backend** y **cómo se conecta con el frontend**. Pensada para que cualquiera del equipo la entienda, sin asumir que ya conocés el código.

1. Qué es y con qué está hecho

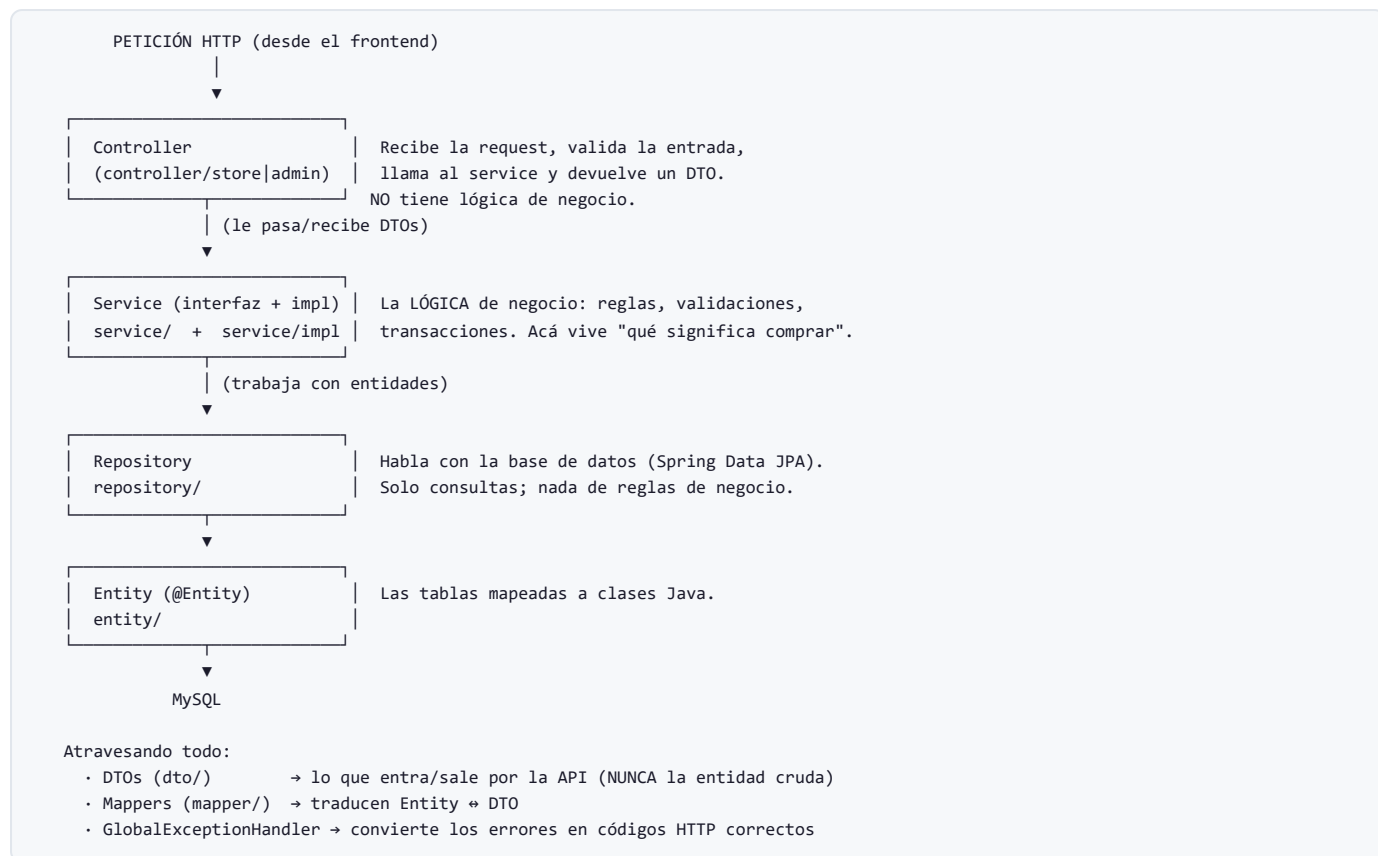
El backend es una **API REST** que le da servicio a la tienda: maneja usuarios, catálogo, carrito, pedidos, stock y reportes. El frontend (React) le pide datos y le manda acciones por HTTP; el backend responde con **JSON**.

Pieza	Tecnología
Lenguaje / framework	Java 17 + Spring Boot 3.3.5
Build	Maven (wrapper <code>./mvnw</code> , no hace falta instalar Maven)
Base de datos	MySQL 8
Migraciones de BD	Flyway (crea y versiona las tablas)
Acceso a datos	Spring Data JPA (Hibernate)
Seguridad	Spring Security + JWT
Reportes	Apache POI (Excel) + OpenPDF (PDF)
Tests	JUnit 5, Mockito, Testcontainers (MySQL real)

Base package del código: `pe.com.krypton` .

2. La idea central: arquitectura por capas

El backend está organizado en **capas**, donde cada capa tiene una sola responsabilidad y **solo puede hablar con la capa de abajo**. Esto mantiene el código ordenado: si algo de la base de datos cambia, no se rompe toda la app; si cambia la API, no se toca la lógica.



Las 4 reglas de oro (no se rompen)

1. **El controller NUNCA toca el repository.** Siempre pasa por el service.
2. **NUNCA se expone una @Entity en la API.** Entran y salen **DTOs**. Si expusieras la entidad, filtrarías datos sensibles (ej. el password) y atarías tu API a la forma de la BD.
3. **Cada service es una interfaz + una implementación.** El controller depende de la interfaz (ej. `ProductoService`), no del `ProductoServiceImpl`. Eso permite testear con mocks.
4. **El checkout es UNA sola transacción** (todo-o-nada): ver flujo en la sección 6.

Nomenclatura (importante para leer el código)

Los **tipos del dominio** están en **spanglish**: nombre en español + sufijo técnico en inglés. Ej: `Usuario`, `Producto`, `Categoria`, `Orden`, `Carrito`, y sus `UsuarioRepository`, `ProductoService`, `OrdenController`, etc. Los **métodos públicos** de services y controllers están en **español** (`registrar`, `listar`, `buscarPorId`, `confirmarCompra` ...).

Quedan en **inglés** (lo exige el framework, no es opcional): los métodos de los repositories (`findByEmail`, `existsBySku` — Spring genera el SQL a partir de ese nombre), los heredados de Spring (`save`, `loadUserByUsername`) y los **nombres de campos** de entidades y DTOs (porque forman el contrato JSON con el frontend — ver sección 7).

3. Recorrido por los paquetes

```
pe.com.krypton
├─ config/           Configuración transversal: SecurityConfig, CorsConfig
├─ security/         JWT: generar/validar token, filtro que intercepta cada request
├─ controller/
│   ├── store/       Endpoints públicos / del cliente (catálogo, carrito, pedidos, auth)
│   └── admin/        Endpoints de gestión (solo ADMIN), bajo /api/admin/**
├─ service/          Interfaces de negocio + base genérica ICRUD
│   └── impl/         Implementaciones (...ServiceImpl) + ICRUDImpl
├─ repository/       Acceso a datos (interfaces Spring Data JPA)
├─ entity/           Entidades @Entity (las tablas) + enums
├─ dto/
│   ├── request/     Lo que ENTRA por la API (ProductoRequest, LoginRequest...)
│   └── response/     Lo que SALE por la API (ProductoResponse, OrdenResponse...)
├─ mapper/           Traduce Entity ↔ DTO (a mano, sin librerías mágicas)
├─ policy/           Reglas de negocio aisladas (EstadoOrdenPolicy: transiciones legales)
├─ spec/             Filtros dinámicos de consultas (JPA Specifications)
├─ report/           Exportadores de PDF y Excel
└─ exception/        Excepciones de dominio + GlobalExceptionHandler
```

La base genérica ICRUD / ICRUDImpl

`service/ICRUD.java` define operaciones CRUD a nivel entidad (`guardar` , `borrar` , `listarTodos` , `obtenerPorId`) y `service/impl/ICRUDImpl.java` las implementa apoyándose en el repository. Los `*ServiceImpl` la **extienden** para reutilizar esa fontanería; hacia afuera siguen exponiendo sus métodos en DTOs. Es código compartido para no repetir el `save` / `findAll` en cada service.

4. Seguridad: JWT de punta a punta

La API es **stateless**: no hay sesión en el servidor. En cada request protegida, el cliente manda un **token JWT** en el header `Authorization` , y el backend lo valida.

```
REGISTRO / LOGIN
POST /api/auth/register {name, email, password} → crea el usuario (201)
POST /api/auth/login   {email, password}       → devuelve { token, tokenType, expiresIn }
```

EN CADA REQUEST PROTEGIDA

El cliente manda: `Authorization: Bearer <token>`

↓

`JwtAuthenticationFilter` intercepta la request, valida el token, carga el usuario (`CustomUserDetailsService`) y lo pone en el contexto de seguridad.

↓

`SecurityConfig` decide si tiene permiso para ese endpoint.

Reglas de acceso (SecurityConfig):

Zona	Quién entra
<code>/api/auth/**</code> , Swagger	Público (cualquiera)
<code>GET /api/products/**</code> , <code>GET /api/categories/**</code> , <code>GET /api/uploads/**</code>	Público (ver catálogo no requiere login)
<code>/api/admin/**</code>	Solo <code>ROLE_ADMIN</code>
Todo lo demás	Cualquier usuario autenticado

El password se guarda con **BCrypt** (hash, nunca en texto plano). El secreto del JWT sale de la variable de entorno `JWT_SECRET` (en dev hay un default; en prod se setea por env).

5. El modelo de datos

8 tablas base (creadas por Flyway en `v1`, extendidas hasta `v8`). Hay reglas que **no son obvias** y son el corazón del negocio:

- **Snapshot de precio:** cuando se hace el checkout, el precio se **copia y congela** en `order_items.unit_price`. El `cart_item` NO guarda precio: muestra el precio VIVO del producto. Así, si después subís el precio, los pedidos viejos conservan el precio al que se compraron.
- **Stock = valor cacheado + kardex:** `products.stock` es la cantidad actual (rápida de leer); `stock_movement` es el historial de movimientos (entradas/salidas). Se actualizan **JUNTOS, en la misma transacción** — nunca uno sin el otro.
- **Claves:** `id` es la PK interna (usada en todas las relaciones). `sku` es la clave de negocio del producto (única, pero nunca se usa como FK).
- **Enums** (`Rol`, `EstadoOrden`, `TipoMovimiento`, `TipoDocumento`, `MetodoPago`) se guardan como texto (`@Enumerated(String)`), no como número, para que la BD sea legible.

Flyway es el **dueño del esquema**: Hibernate solo valida (`ddl-auto: validate`). Nunca se edita una migración ya aplicada — se agrega una nueva `v{n+1}`.

6. Flujos completos (paso a paso)

Registrarse y entrar: el front pide `POST /api/auth/register`, luego `POST /api/auth/login`, guarda el token, y a partir de ahí lo manda en cada request.

Ver el catálogo: `GET /api/products` con filtros (nombre, categoría, rango de precio) y paginación. Es público, no requiere token.

Carrito: el carrito vive en la BD por usuario. `POST /api/cart/items` agrega, `PUT /api/cart/items/{id}` cambia cantidad, `DELETE` quita.

Checkout (el flujo crítico — `OrdenServiceImpl.confirmarCompra`, todo en UNA transacción):

```
POST /api/orders/checkout {documentType, customerName, customerDoc}
1. Lee el carrito del usuario.
2. Crea la Orden + sus ItemOrden (copiando el precio = snapshot).
3. Registra una SALIDA en stock_movement por cada producto.
4. Descuenta products.stock.
5. Vacía el carrito.
→ Si CUALQUIER paso falla, se revierte TODO (no queda una orden a medias).
```

Pagar: `POST /api/orders/{id}/pay` simula el pago y mueve el estado `PENDIENTE` → `CONFIRMADA`. Las transiciones de estado las controla `EstadoOrdenPolicy` (ej. no se puede ir de `ENTREGADO` a `PENDIENTE`). El comprobante se descarga como PDF: `GET /api/orders/{id}/comprobante`.

Reportes (admin): ventas por período, top de productos, kardex — con export a Excel/PDF.

7. Cómo se conecta con el frontend

Esta es la parte clave: **el backend y el frontend son dos proyectos separados** que se hablan **solo por HTTP/JSON**. El backend no sabe nada de React; el front no sabe nada de Java.

El "contrato" que los une

1. **La URL base.** El front apunta al backend en `http://localhost:8080` (configurable con `VITE_API_BASE_URL`). En prod sería la URL pública del backend.
2. **CORS.** Como el front (`localhost:5173`) y el back (`localhost:8080`) están en orígenes distintos, el navegador bloquea las llamadas salvo que el backend lo permita. `CorsConfig` habilita explícitamente el origen `http://localhost:5173`, los métodos `GET/POST/PUT/PATCH/DELETE`, los headers `Authorization` y `Content-Type`, y expone `Content-Disposition` (para que el front pueda leer el nombre del archivo en las descargas de PDF/Excel).

3. **El token JWT.** Tras el login, el front guarda el token y lo manda en `Authorization: Bearer <token>` en cada request. El backend lo valida en cada una.
4. **Los nombres de los campos JSON.** Acá está la sutileza más importante:

El **nombre de la clase Java** no viaja por la red. Lo que viaja son los **nombres de los campos**. Jackson (el serializador) convierte `ProductoResponse` a JSON usando sus campos: `{ "id":..., "sku":..., "name":..., "price":... }`. El front tiene su propia interfaz TypeScript (que llama `ProductoResponse`) con **esos mismos campos**. Coinciden los **campos**, no los nombres de clase.

Por eso el refactor del backend (que renombró clases y métodos a español/spanglish) **no afectó al frontend**: los campos de los DTOs siguen en inglés, las URLs siguen iguales, así que el JSON es idéntico. El front no se tocó.

Mapa de endpoints (lo que el frontend consume)

Públicos / cliente (`controller/store`)

Método	Ruta	Para qué
POST	<code>/api/auth/register</code>	Crear cuenta
POST	<code>/api/auth/login</code>	Entrar (devuelve token)
GET	<code>/api/products</code>	Catálogo paginado + filtros
GET	<code>/api/products/{id}</code>	Detalle de producto
GET	<code>/api/categories</code>	Listar categorías
GET	<code>/api/uploads/images/{file}</code>	Servir imágenes de productos
GET	<code>/api/cart</code>	Ver el carrito (autenticado)
POST/PUT/DELETE	<code>/api/cart/items...</code>	Modificar el carrito
POST	<code>/api/orders/checkout</code>	Confirmar compra
GET	<code>/api/orders</code> , <code>/api/orders/{id}</code>	Mis pedidos
POST	<code>/api/orders/{id}/pay</code>	Pagar un pedido
GET	<code>/api/orders/{id}/comprobante</code>	Descargar comprobante PDF

Gestión (`controller/admin` , solo ADMIN)

Ruta base	Operaciones
<code>/api/admin/products</code>	Crear, editar, eliminar productos
<code>/api/admin/products/{id}/images</code>	Subir / reordenar / portada / borrar imágenes
<code>/api/admin/categories</code>	CRUD de categorías
<code>/api/admin/orders</code>	Listar, ver, cambiar estado, comprobante
<code>/api/admin/users</code>	Listar, crear, cambiar rol / estado
<code>/api/admin/reports/...</code>	Ventas, top productos, kardex + export Excel/PDF

Cómo se reportan los errores

El backend no devuelve stacktraces. El `GlobalExceptionHandler` traduce cada error a su código HTTP y un mensaje legible (`{ "status":..., "error": "mensaje" }`):

Situación	HTTP
Validación de entrada falla (<code>@Valid</code>)	400 Bad Request
Recurso no existe	404 Not Found
Regla de negocio (ej. SKU duplicado, categoría en uso)	409 Conflict

Situación	HTTP
Transición de estado inválida	422 Unprocessable Entity
Sin token / token inválido	401 Unauthorized
Sin permisos (no-ADMIN en /admin)	403 Forbidden

El front lee ese `error` y lo muestra al usuario.

8. Cómo levantarlo y testearlo

```
# 1. Base de datos (desde la raíz del repo)
docker compose up -d                # MySQL 8 en localhost:3307

# 2. Backend
cd backend && ./mvnw spring-boot:run  # Flyway crea tablas + siembra datos → http://localhost:8080

# Tests
./mvnw test                          # toda la suite (incluye integración con Docker)
./mvnw test -Pfast                   # rápido: omite los de integración (sin Docker)
```

Usuario admin sembrado: `admin@krypton.pe` / `Admin123!` .

Más detalle de arquitectura: [arquitectura-backend.md](#) . Modelo de datos: [modelo-datos.md](#) .